A Type Discipline for the Reduction Stages

Indexed problem-types, carried decoders, and a soundness theorem for colouring → IS → gadget → unit-disk layout → device

Design sketch — companion to the paper and the Haskell harness

June 3, 2026

Abstract

This is a sketch of a type system aimed at *one thing only*: making the correctness of the compilation *stages* static. Ordinary programming errors — scope, arity, base types, totality of plain functions — are **not** our concern; those are delegated to a standard Hindley–Milner core with refinements (Sec. 1). What we add on top is a small *certified-reduction calculus*: every stage is typed as an answer-preserving morphism that *carries an executable decoder*, indices track the invariants the paper proves ($\alpha(G') = n$, $\mathrm{MWIS}(A) = \alpha + C$, exact unit-disk realizability, device budget), and a single composition rule yields the end-to-end soundness theorem. The geometry stage is given a deliberately *weaker* type when it leaves false edges, which is exactly the honest separation "heuristics affect feasibility, never the answer." The whole discipline is the *static shadow* of the runtime oracle already in `code/`.

Contents

1 Scope: what we delegate, what we invent

Delegated (adopt off the shelf). Variables, binding, function arity, products/sums, plain base types ($\mathbb{Z}$, lists, finite maps, the `Graph` carrier), and totality of ordinary functions are handled by a standard typed core — take Hindley–Milner with a refinement/SMT layer (in the style of liquid types). We write $\Gamma \vdash e : T$ for that judgment and never re-derive it.

Invented here (the subject of this note). A second judgment form for *transformation stages*. Its types are not ordinary data types but *problem-types* — "an instance of k-colouring on n vertices," "a weighted-IS instance with offset C," "a unit-disk instance that fits the device" — and its arrows are *certified reductions* carrying decoders. This is the part that turns the paper's theorems into typing rules.

Key point. The slogan: *base types say a program is well-formed; problem-types say a* ***reduction*** *is faithful.* We only build the second.

2 Problem-types (the sorts)

A *problem-type* names a class of decision/optimisation instances together with the indices we must track to state preservation. Let G range over input graphs, κ over colourings, S over (weighted) vertex sets, m over device bitstrings.

$A, B \;::=$	$\mathsf{Col}\langle n, k\rangle$	k-colouring instance, $n = \lvert V\rvert$
	$\mathsf{Col}\langle n, k \mid \beta\colon B\rangle$	boundary-precoloured (β proper on $B \subseteq V$)
	$\mathsf{IS}\langle N\rangle\{\, P \,\}$	independent-set instance, N nodes, refinement P
	$\mathsf{WIS}\langle N,\, C\rangle$	weighted IS, N nodes, additive offset C
	$\mathsf{UDG}\langle N,\, r,\, d\rangle$	*exactly* realised unit-disk inst. (radius r, spacing d)
	$\widetilde{\mathsf{UDG}}\langle N,\, r,\, d \mid f\rangle$	*approximately* realised, f residual false edges
	$\mathsf{Dev}\langle N\rangle \quad (N \le N_{\max})$	device-feasible instance

Each problem-type A comes with a *solution domain* $\mathrm{Sol}(A)$ (colourings, vertex sets, bitstrings) and an *optimum* $\mathrm{opt}(A) \in \mathbb{R} \cup \{\bot\}$. The refinement P on IS is propositional and is where Stage-1 exactness lives: the canonical one is

$$P_{\mathrm{s1}} \;\equiv\; (\alpha = n \iff G \text{ is } k\text{-colourable}) \;\wedge\; \alpha \le n.$$

Remark 2.1 (Why $\widetilde{\mathsf{UDG}}$ exists)**.** UDG is the placement in which the realised unit-disk graph equals the *prescribed* gadget adjacency *exactly*. When the layout optimiser leaves false edges (the generic case before Nguyen gadgets), the object is honestly a *different* graph, so it gets a different type, $\widetilde{\mathsf{UDG}}$, and — crucially — $\widetilde{\mathsf{UDG}}$ **is not a legal source for the exact decoder** (Sec. 4). The type is what stops us from silently reading an answer off a corrupted layout.

3 The certified-reduction judgment

Definition 3.1 (Decoder)**.** A *decoder* from B to A is a partial map $\delta : \mathrm{Sol}(B) \rightharpoonup \mathrm{Sol}(A)$. It is *total on optima*, written $\mathsf{tot\text{-}opt}(\delta)$, if δ is defined on every $s \in \mathrm{Sol}(B)$ with value $\mathrm{opt}(B)$ and maps it to a value-$\mathrm{opt}(A)$ solution of A.

Definition 3.2 (Certified reduction)**.** The judgment

$$\vdash\; \tau \;:\; A \overset{\delta}{\rightsquigarrow} B$$

asserts: τ is a transformation taking an instance of A to an instance of B, δ is a decoder $B \rightharpoonup A$ with $\mathsf{tot\text{-}opt}(\delta)$, and

$$\text{(answer preservation)} \qquad \mathrm{opt}(A) \text{ is recoverable as } \delta(s^\star) \text{ for any optimum } s^\star \text{ of } B.$$

Equivalently: *solve B on the device, apply δ, get the answer to A.*

This single arrow is the backbone. Everything else either *introduces* an arrow for one stage (Sec. 4), *composes* arrows (Sec. 5), or *refines* the discipline with extra indices (safety, budget, decomposition).

4 Per-stage typing rules

In each rule the premises above the line are the *proof obligations*; in the implementation they are discharged either by a once-and-for-all proof or by the brute-force oracle in `code/` (Sec. 9).

Stage 1 — colour-choice. The standard colouring→IS reduction, indices recording $N = nk$ and the exactness refinement.

$$\frac{n = |V(G)| \qquad k \geq 1}{\vdash\ \mathsf{colorChoice}_k\ :\ \mathsf{Col}\langle n, k\rangle \overset{\mathsf{decodeColoring}}{\rightsquigarrow} \mathsf{IS}\langle nk\rangle\{P_{\mathrm{s1}}\}} \quad \text{S1}$$

decodeColoring is total on size-n independent sets (one slot per vertex) and partial elsewhere — exactly tot-opt.

Stage 2 — gadget. The weighted-IS embedding with its *static* offset.

$$\frac{C = 2\,|E(G')|}{\vdash\ \mathsf{gadgetize}\ :\ \mathsf{IS}\langle N\rangle\{P\} \overset{\mathsf{decodeLogical}}{\rightsquigarrow} \mathsf{WIS}\langle N + 2|E(G')|,\ C\rangle} \quad \text{S2}$$

The offset C is carried in the type so "subtract C" is type-directed; the invariant MWIS $= \alpha + C$ is the obligation. decodeLogical (keep logical atoms) is tot-opt.

Stage 3 — layout. Here the type *branches* on realizability. Let p be a placement and r a radius, and write $\mathsf{realizes}(p, r)$ for "the unit-disk graph of (p, r) has edge set exactly E_A" and $\mathsf{miss}(p, r) = \varnothing$, $\mathsf{false}(p, r)$ for the residual false edges.

$$\frac{\mathsf{realizes}(p, r) \qquad \mathsf{spacing}(p) \geq d}{\vdash\ \mathsf{layout}(p, r)\ :\ \mathsf{WIS}\langle N, C\rangle \overset{\mathrm{id}}{\rightsquigarrow} \mathsf{UDG}\langle N, r, d\rangle} \quad \text{S3-EXACT}$$

$$\frac{\mathsf{miss}(p, r) = \varnothing \qquad f = |\mathsf{false}(p, r)| > 0}{\vdash\ \mathsf{layout}(p, r)\ :\ \mathsf{WIS}\langle N, C\rangle \rightsquigarrow \widetilde{\mathsf{UDG}}\langle N, r, d \mid f\rangle} \quad \text{S3-APPROX}$$

Note S3-EXACT carries the identity decoder (geometry never relabels a solution) and lands in UDG; S3-APPROX carries *no* certified decoder and lands in $\widetilde{\mathsf{UDG}}$. The only way to turn $\widetilde{\mathsf{UDG}}$ back into a legal UDG is to interpose the metric repair transform that re-routes false edges through copy / crossing gadgets,

$$\frac{}{\vdash\ \mathsf{nguyen}\ :\ \widetilde{\mathsf{UDG}}\langle N, r, d \mid f\rangle \overset{\mathsf{decodeRoute}}{\rightsquigarrow} \mathsf{WIS}\langle N', C'\rangle} \quad \textsc{Repair}$$

after which S3-EXACT can be attempted again. *This is the type-level form of the deferred engineering fork: without* Repair*, a false-edged layout simply does not type at the exact level.*

Emit — device budget. Crossing onto hardware needs the atom count to fit.

$$\frac{N \leq N_{\max}}{\vdash\ \mathsf{emit}\ :\ \mathsf{UDG}\langle N, r, d\rangle \overset{\mathrm{id}}{\rightsquigarrow} \mathsf{Dev}\langle N\rangle} \quad \textsc{Fit}$$

Key point. The pipeline can reach Dev **only through** UDG, never through $\widetilde{\mathsf{UDG}}$. So "the machine measured a bitstring" is connected to "a correct colouring" by a *chain of* tot-opt *decoders* — or it is not connected at all, and the type checker says so.

5 Composition and the soundness theorem

$$\frac{\vdash\ \tau_1 : A \overset{\delta_1}{\rightsquigarrow} B \qquad \vdash\ \tau_2 : B \overset{\delta_2}{\rightsquigarrow} C}{\vdash\ \tau_2 \circ \tau_1\ :\ A \overset{\delta_1 \circ \delta_2}{\rightsquigarrow} C}\ \textsc{Comp}$$

Theorem 5.1 (Pipeline soundness)**.** *If*

$$\vdash\ \Pi\ :\ \mathsf{Col}\langle n, k\rangle \overset{\delta}{\rightsquigarrow} \mathsf{Dev}\langle N\rangle \quad \textit{is derivable (so } N \le N_{\max} \textit{ and every layer realised } \text{exactly}\textit{),}$$

then for any device measurement m with value $\mathrm{opt}(\mathsf{Dev}\langle N\rangle)$*, the decoded $\delta(m)$ is a proper k-colouring of G if one exists, and otherwise the pipeline reports "not k-colourable" soundly.*

Proof sketch. By induction on the derivation using Comp: each arrow is answer-preserving with a tot-opt decoder, and the composite of tot-opt decoders is tot-opt, so $\delta = \delta_{\mathrm{s1}} \circ \delta_{\mathrm{s2}} \circ \mathrm{id} \circ \mathrm{id}$ maps an optimum bitstring back through WIS (strip offset C), IS (read one slot per vertex), to a colouring; the Stage-1 refinement P_{s1} supplies the iff. The only non-identity geometric step is admitted by S3-exact, whose premise $\mathsf{realizes}(p, r)$ guarantees the realised graph *is* A, so its optima coincide. □ □

Corollary 5.2 (Ill-ordered pipelines are ill-typed)**.** *The following do not type, matching the error catalogue (`errors.tex` #6/#10, pipeline-stage ordering):*

- gadgetize *before* colorChoice*: S2 expects* IS*, is given* Col *— no rule applies.*
- emit *before* layout*: Fit expects* UDG*, is given* WIS*.*
- *reading an answer off a false-edged layout: there is no* tot-opt *decoder out of* $\widetilde{\mathsf{UDG}}$ *(only Repair, which goes* back *to* WIS*).*

6 Refinement A: certificate grade (witness-relative vs. true χ)

The paper's vocabulary distinguishes a cheap, machine-checkable certificate from a proof-only one. We expose this as a *grade* $g \in \{\mathsf{w}, \chi\}$ on colouring conclusions, with $\mathsf{w} \sqsubseteq \chi$:

- $\mathsf{Col}\langle n, k\rangle^{\mathsf{w}}$ — *witness-relative*: a carried colouring κ certifies $\chi(G) \le |\kappa|$ in poly time. Default.
- $\mathsf{Col}\langle n, k\rangle^{\chi}$ — *exact*: $\chi(G) = k$ as a discharged proof obligation. Stronger, proof-only.

All stage rules are grade-polymorphic and *preserve* the grade; only an explicit prove_{χ} step (a discharged lower-bound proof, e.g. a clique or an exhaustive small-case check) lifts $\mathsf{w} \Rightarrow \chi$. Theorem 5.1 holds at grade w already — which is the point: *soundness of the compilation does not require knowing the true chromatic number.*

7 Refinement B: the safety effect (optional supergraph path)

The *secondary* (supergraph) path rewrites G on the *same* vertices by adding edges, decoded by restriction. Such a transform may inflate χ, so it carries a *safety effect* s:

$$\vdash\ t\ :\ \mathsf{Col}\langle n, k\rangle \overset{\mathsf{restrict}}{\underset{s}{\rightsquigarrow}} \mathsf{Col}\langle n, k'\rangle, \qquad s \in \{\mathsf{safe},\ \mathsf{asafe}(c)\}.$$

- safe: χ preserved (poly-time, $\chi(\text{target}) = \chi(\text{source})$).
- $\mathsf{asafe}(c)$: never lowers χ, and $\chi(\text{target}) \le \chi(\text{source}) + c$.

Effect monoid (composition).

$$\mathsf{safe} \oplus \mathsf{safe} = \mathsf{safe}, \qquad \mathsf{safe} \oplus \mathsf{asafe}(c) = \mathsf{asafe}(c), \qquad \mathsf{asafe}(c) \oplus \mathsf{asafe}(c') = \mathsf{asafe}(c + c').$$

The composition rule combines decoders *and* effects:

$$\frac{\vdash t_1 : A \overset{\delta_1}{\underset{s_1}{\rightsquigarrow}} B \qquad \vdash t_2 : B \overset{\delta_2}{\underset{s_2}{\rightsquigarrow}} C}{\vdash t_2 \circ t_1 : A \overset{\delta_1 \circ \delta_2}{\underset{s_1 \oplus s_2}{\rightsquigarrow}} C} \quad \textsc{Comp-S}$$

Mandatory precondition (the impossibility theorem, as a side-condition). A $\mathsf{safe}/\mathsf{asafe}(c)$ introduction is *only* derivable under the neighbourhood bound $\sigma(G) = \max_v \alpha(G[N(v)]) \le 5$; otherwise the only available type is unsafe (no χ certificate). This is precisely the paper's $\chi(K_{1,n}) = 2$ vs. $\chi(H) \ge \lceil n/5 \rceil$ obstruction, refusing to type as the type system.

Remark 7.1 (Answering "does transform order affect safety?")**.** Because $\oplus$ sums the bumps c but the *carrier graph* differs between orders, two orderings of the same multiset of $\mathsf{asafe}(\cdot)$ transforms are type-equal only if their accumulated bounds and side-conditions both match. The type therefore *records* order-sensitivity as a difference in the derived effect — turning the open question of `questions.md` into a checkable property rather than a conjecture.

8 Refinement C: decomposition (precolour / split / stitch)

Decomposition is where the discipline becomes *substructural*: pieces are resources, and a shared separator is a value two pieces must *agree* on.

Precolour. Pinning a proper partial colouring β on B yields a constrained instance; the precoloured colour-choice graph is smaller.

$$\frac{\beta : B \to [k] \text{ proper partial on } G}{\vdash\ \mathsf{pin}_\beta : \mathsf{Col}\langle n, k\rangle \overset{\mathsf{ext}_\beta}{\rightsquigarrow} \mathsf{Col}\langle n, k \mid \beta\rangle} \quad \textsc{Pre}$$

$$\frac{}{\vdash\ \mathsf{colorChoice}_k : \mathsf{Col}\langle n, k \mid \beta{:}B\rangle \overset{\mathsf{decodeColoring}_\beta}{\rightsquigarrow} \mathsf{IS}\langle k(n - |B|) + |B|\rangle\{P_{\mathrm{s1}}\}} \quad \text{S1-}\beta$$

Split (resource-bounded). A separator S with pieces $G_1, \dots, G_p$ ($G_i = G[S \cup C_i]$, width $w = \max_i |G_i|$) produces a *context* of subgoals, each of which must individually fit the device.

$$\frac{S \text{ separates } G \qquad w = \max_i |G_i| \qquad (wk)^2 \le N_{\max}}{\vdash\ \mathsf{split}_S : \mathsf{Col}\langle n, k\rangle \implies \{\, \mathsf{Col}\langle |G_i|, k \mid \beta_i{:}S\rangle \,\}_{i=1}^{p}} \quad \textsc{Split}$$

The premise $(wk)^2 \le N_{\max}$ is the typed budget: *peak device size depends on w, not n*, so each piece is compiled by the primary pipeline to its own $\mathsf{Dev}\langle N_i\rangle$ with $N_i \le N_{\max}$. This is register spilling, for atoms.

Stitch (agreement). Piece solutions combine *iff* they agree on shared vertices.

$$\frac{\forall i\ \kappa_i \text{ proper on } G_i \qquad \forall i, j \text{ sharing } S :\ \kappa_i|_S = \kappa_j|_S}{\vdash\ \mathsf{stitch} : \{\kappa_i\}_i \overset{\bigcup}{\rightsquigarrow} \kappa \text{ proper on } G} \quad \textsc{Stitch}$$

For a *clique* separator the agreement premise is discharged *for free* by a relabelling coercion (no enumeration):

$$\frac{S \text{ a clique} \qquad k \ge |S| \qquad \kappa_i, \kappa_j \text{ proper}}{\exists \pi \in \mathrm{Sym}([k]) :\ (\pi \circ \kappa_j)|_S = \kappa_i|_S} \quad \textsc{Realign}$$

For a general (non-clique) separator the agreement is found by a *bounded search* over the boundary table — at most $k^{|S|}$ candidate β — an effect that the type records as the cost $O(k^{|S|})$ the user already pays in `decomposition.tex`.

Theorem 8.1 (Decomposed soundness)**.** *If* Split *produces pieces each compiled by a derivation of Theorem 5.1, and* Stitch *applies (agreement holds, by* Realign *in the clique case), then* $\kappa = \bigcup_i \kappa_i$ *is a proper* k*-colouring of* G*, using peak device size* $O((wk)^2) \le N_{\max}$*.*

9 The discipline is the static shadow of your oracle

Every rule corresponds to a function already in (or planned for) `code/`, and every premise to a property the brute-force oracle checks. The type system is what you get by promoting those runtime checks to static obligations.

rule	code artefact	obligation checked by
S1	`Stage1.colorChoiceGraph`, `decodeColoring`	oracle: $\alpha(G') = n \iff k$-col
S2	`Gadget.gadgetize`, `gOffset`, `decodeLogical`	oracle: $\mathrm{MWIS}(A) = \alpha + C$
S3-exact	`Layout.layoutGadget`, `minFeasibleRadius`	`missingEdges`$= \varnothing$
S3-approx	`Layout.falseEdges`	$\lvert\mathsf{falseEdges}\rvert = f > 0$
Fit	`order ag` vs. $N_{\max}$	atom-count comparison
Pre, S1-β	`colorChoiceGraphPrecolored`	oracle (precoloured exactness)
Split, Stitch	decomposition driver *(to build)*	agreement on S; budget $(wk)^2 \le N_{\max}$

Key point. Implementation path: keep the oracle as the *runtime* witness, and introduce each typing rule as a refinement obligation that, when the proof is absent, *falls back* to the oracle on small instances. Mechanise S1, S2, Comp, and Theorem 5.1 first (in Lean/Coq, or as Liquid refinements); they are the smallest closed core that already yields end-to-end faithfulness.

Minimal core to build first

1. Problem-types $\mathsf{Col}, \mathsf{IS}, \mathsf{WIS}, \mathsf{UDG}, \widetilde{\mathsf{UDG}}, \mathsf{Dev}$ with their indices (Sec. 2).
2. The arrow $A \overset{\delta}{\rightsquigarrow} B$ with tot-opt decoders, rules S1, S2, S3-exact, Fit, Comp (Secs. 3–5).
3. Theorem 5.1 (composition of decoders) — the payoff.
4. Then layer in $\widetilde{\mathsf{UDG}}$/Repair (honesty about geometry), the safety effect (supergraph path), and decomposition.

Everything beyond step 3 is optional polish; steps 1–3 already make "the measured bitstring decodes to a correct colouring" a *typed* guarantee — which is the certified-compilation thesis in its smallest form.